

Our Approach: TomaNet

A Plain-English Guide to the Model We Built

Tomato disease detection project

September 11, 2026

Contents

1	What This Project Is	2
2	The Model: TomaNet-C and TomaNet-D	2
3	What Makes Our Design Different	2
4	Datasets We Actually Use	3
4.1	For Classification (“what disease is this leaf?”)	3
4.2	For Detection (“draw a box around every disease spot on this plant”)	3
4.3	An Important Data-Quality Fix We Made	3
5	Honest Limits – What We Have Not Done Yet	3

1 What This Project Is

Tomato plants get sick from things like bacterial spot, blight, and viruses. A farmer or a phone app can either:

- look at a photo of *one leaf* and say *what disease this is* – we call this **classification**, or
- look at a photo of a *whole plant* and **draw a box** around every diseased spot it can find, even several at once, on several leaves – we call this **detection**.

We built one small, efficient model design that does both jobs, and the data pipeline needed to train and check it honestly.

2 The Model: TomaNet-C and TomaNet-D

Both are built from the same reusable building block, called TOMABLOCK. Think of it like a Lego brick: the exact same brick is used to build the “trunk” – the part that looks at the raw image and figures out what’s in it – for *both* tasks.

- **TomaNet-C** (classification) = trunk → one small extra layer that turns the trunk’s output into a single disease name.
- **TomaNet-D** (detection) = the exact same trunk → a “neck” that combines information at different zoom levels → a head that draws boxes.

Because the trunk is identical – not just similar, literally the same list of layers, checked automatically by an automated test – a trunk trained on one task can jump-start training on the other. This is not a marketing claim; it is a structural fact about the code.

Both come in three sizes, so the same design scales from a phone to a more capable server. These are the real, measured numbers (not estimates) from actually running our model-building script:

	n (smallest)	s	m (largest)
TOMANET-C (classification)	1.32 M params	4.47 M params	14.44 M params
TOMANET-D (detection)	2.92 M params	10.64 M params	30.79 M params

(“M” = million. For comparison, popular phone-sized detectors are usually in the same 1–5 M range.)

3 What Makes Our Design Different

We are not claiming to have invented brand-new ideas from scratch. We took three good, published ideas and combined them in a way nobody had applied to tomato disease work before, and we picked each one for a specific, explainable reason.

1. **We only convolve a quarter of the channels at a time (“PConv”).** A normal convolution touches every channel of every layer, which is slow mainly because of memory movement, not the actual math – and memory movement is the real speed bottleneck on small devices like a phone. This idea comes from a method called FasterNet [1]. We use it so the model stays fast and light.
2. **We add a positional attention step (“Coordinate Attention”).** Disease spots on a leaf are small and appear in specific places, not spread evenly across the whole leaf. A common shortcut used in many lightweight models (channel-only attention) throws away exactly that positional information. Coordinate Attention [2] keeps track of *where* on the leaf something unusual is happening, which matters for small, localised problems like lesions.
3. **We train with a richer version of the block, then simplify it before deployment (“re-parameterisation”).** During training, our block briefly has three small parallel paths, which helps it learn better. Right before the model is used for real, those three paths are mathematically folded back into one single, simple path – so training gets the benefit, but the final deployed model pays no extra cost. This RepVGG-style trick [3] is checked in our own tests to be numerically exact (the folded and unfolded versions agree to about five decimal places).

4. **One trunk, two jobs.** As described above, the same trunk serves both classification and detection, byte-for-byte identical in the model definition. This is the part of the design that is genuinely uncommon in tomato disease research specifically – most published work builds a completely separate model for each task.

We are honest about scale: our detection model is not smaller than every competing lightweight detector on the market. The point we are actually trying to prove is that one small, shared design can do both jobs well and transfer knowledge between them – and that claim has to be measured with real experiments, not assumed.

4 Datasets We Actually Use

Many public tomato datasets exist. Only the ones below are wired into working code right now – meaning there is a script that downloads them, cleans them, and prepares them for training. Nothing else is included here; this list is deliberately limited to what actually runs today.

4.1 For Classification (“what disease is this leaf?”)

- **PlantVillage** – 18,160 tomato leaf photos, 10 disease/healthy classes. Taken in a lab on a plain background, not in a real field.
- **Taiwan tomato dataset** – 622 real field photos, 6 classes. Small, but real-world, so it is our first sanity check for any new model.
- **PlantDoc** – around 700 tomato photos scraped from the web, 9 classes.
- **CCMT** – field photos from Ghana, 5 classes.
- **PlantVillage (Zenodo re-split)** – a smaller, pre-split copy of PlantVillage, used as a quick stand-in when a full run isn’t needed.

4.2 For Detection (“draw a box around every disease spot on this plant”)

- **Tomato-Village (detection variant)** – field photos from Rajasthan, India. This is currently our *only* working detection dataset. After preparing it, we have 14,259 usable images with 6 disease classes that actually have boxes drawn on them. (Two of the eight label names in the source data turned out to have no boxes anywhere in the files – a real property of the raw data, confirmed by checking every label file directly, not a mistake in our code.)

No other public dataset has a working detection adapter yet. We checked one strong candidate, PlantDoc, and found that its public copy on GitHub does not actually contain any box annotation files, despite being listed elsewhere as box-annotated – so it is classification-only in practice, at least from that source.

4.3 An Important Data-Quality Fix We Made

The Tomato-Village detection data ships with its own train/test split already done by its authors – but we found that split leaks: many photos in their “test” portion are slightly-edited duplicates of photos in their “training” portion (rotated, recoloured, etc.), which would make any accuracy number misleadingly high. We wrote our own, corrected split that keeps every edited copy of the same original photo together on one side, so the test numbers we report are trustworthy.

5 Honest Limits – What We Have Not Done Yet

What exists today is: a working, tested data pipeline for one classification job and one detection job, a shared model architecture proven to build correctly at all six sizes, and one real (if short) training run on each task confirming the whole pipeline works end to end. Larger training runs, comparisons against other models, and the deeper research questions (cross-domain generalisation, explainability,

unknown-disease detection) are the next steps – see the companion document, *Research Context*, for what the literature says about those questions and how they shape our plan.

References

- [1] J. Chen et al., “Run, don’t walk: partial convolution (FasterNet),” *CVPR*, 2023. arXiv:2303.03667
- [2] Q. Hou, D. Zhou, J. Feng, “Coordinate attention for efficient mobile network design,” *CVPR*, 2021. arXiv:2103.02907
- [3] X. Ding et al., “RepVGG: making VGG-style ConvNets great again,” *CVPR*, 2021. arXiv:2101.03697